C-OWPF Codebase Reference

A Complete, Detailed Summary of the Python Implementation
Optimal Water Flow (OWF) and Coupled Optimal Water–Power Flow (C-OWPF)

Companion documentation to IEEE Access-2024-18604

August 8, 2026

Abstract

This document is a complete, self-contained reference for the Python implementation of the C-OWPF solver. Part I gives the architecture and the core data structures. Part II explains, in detail, *how* each optimization problem is solved — the decoupled Optimal Water Flow (OWF, problem P3) and the coupled Optimal Water–Power Flow (C-OWPF, problem P2) — naming the exact function call chain, the variables passed in, and the outputs produced at each stage. Part III is an auto-generated per-module API reference documenting every function and class (200 functions, 20 classes across 46 modules): its signature, its inputs, and its outputs. The reference is generated directly from the source by an AST walk, so it is guaranteed to match the code. Variable-speed pumps (VSP) and pressure-reducing valves (PRV) are covered in Section 6. This revision also covers the commercial-solver support (MOSEK/Gurobi repo-local license bootstrap), the teaching features (per-iteration constraint viewer, network inspector, pump curves, PV capacity utilization), and the water→DSO schedule-transmit workflow: the Water tab publishes the EPANET rule-based and/or C-OWF-optimized pump schedules, the Power tab acknowledges and solves one OPF per schedule, and the Coupled tab compares C-OWPF against both decoupled practices (EPANET rules + OPF and C-OWF + OPF) with per-practice savings. New in this revision: the coupled VSP solves finish with the same speed-pinned EPANET replay polish as the decoupled path (`coupled.runner._replay_polish`; tens of ft of relaxation-gap replay error → ∼0.005 ft), the `ui/pump_power.py` panel compares the optimizer's $\sum_t$ *linearized* pump power against the *true* nonlinear power in all three tabs (`CoupledResult` now carries `ppump_linear`), and the Coupled tab renders the same water-side plot suite as the Water tab (map, flow animation, schedule with speeds, flows/heads vs the EPANET replay, convergence, error).

Contents

I Architecture and Core Data Structures **4**

1 Package layout **4**

2 The solver stack **4**

3 Core data structures **4**

II How the Problems Are Solved **6**

4 Decoupled Optimal Water Flow (OWF, problem P3) **6**
4.1 Entry point and inputs . . . 6
4.2 Building the problem: `owf.network.setup` . . . 6
4.3 The successive-linearization loop: `owf.solver.solve_owf` . . . 6
4.4 The four modes (how the schedule is obtained) . . . 7
4.5 Validation: `owf.validation.validate_schedule` . . . 7

5 Coupled Optimal Water–Power Flow (C-OWPF, problem P2) **7**
5.1 Entry point and inputs . . . 7
5.2 One coupled solve: `coupled.coupled_lp.solve_coupled` . . . 7

5.3 The voltage-aware schedule search: coupled.schedule.optimize_coupled_schedule . . . 8
5.4 Validation: coupled.validation.validate_coupled . . . 8

6 Variable-Speed Pumps and Pressure-Reducing Valves 8
6.1 VSP (variable-speed pumps) . . . 8
6.2 PRV (pressure-reducing valves) . . . 9
6.3 Related coupled drivers . . . 9

III Complete Per-Module API Reference 10

7 Entry points 10
7.1 main_owf.py . . . 10
7.2 app.py . . . 11
7.3 guide.py . . . 11

8 owf – Water Distribution Network & OWF solver 11
8.1 owf/__init__.py . . . 11
8.2 owf/config.py . . . 11
8.3 owf/epanet_io.py . . . 12
8.4 owf/connection_matrices.py . . . 14
8.5 owf/network.py . . . 14
8.6 owf/initial_values.py . . . 16
8.7 owf/linearization.py . . . 17
8.8 owf/constraints.py . . . 18
8.9 owf/solver.py . . . 20
8.10 owf/warmstart.py . . . 21
8.11 owf/validation.py . . . 23
8.12 owf/plots.py . . . 24
8.13 owf/netmap.py . . . 25

9 pdn – Power Distribution Network models 25
9.1 pdn/__init__.py . . . 25
9.2 pdn/feeders.py . . . 25
9.3 pdn/feeders_paper.py . . . 25
9.4 pdn/network.py . . . 25
9.5 pdn/lindistflow.py . . . 26
9.6 pdn/powerflow.py . . . 27
9.7 pdn/opf.py . . . 27

10 coupled – Coupled C-OWPF solver 28
10.1 coupled/__init__.py . . . 28
10.2 coupled/config.py . . . 28
10.3 coupled/coupled_lp.py . . . 28
10.4 coupled/runner.py . . . 30
10.5 coupled/schedule.py . . . 30
10.6 coupled/validation.py . . . 31

11 ui – Streamlit application 31
11.1 ui/__init__.py . . . 31
11.2 ui/theme.py . . . 31
11.3 ui/landing.py . . . 32
11.4 ui/water_section.py . . . 32
11.5 ui/power_section.py . . . 33
11.6 ui/coupled_section.py . . . 33
11.7 ui/pdn_plots.py . . . 34
11.8 ui/prv_plots.py . . . 35

11.9 ui/constraint_view.py . . . 35
11.10ui/correlations.py . . . 35
11.11ui/capture.py . . . 36
11.12ui/pump_power.py . . . 36

12 tests **37**
12.1 tests/test_coupled.py . . . 37
12.2 tests/test_eightnode.py . . . 37
12.3 tests/test_more_networks.py . . . 38
12.4 tests/test_optimize_schedule.py . . . 39
12.5 tests/test_plots.py . . . 39

Part I. Architecture and Core Data Structures

1 Package layout

The implementation lives under `Python/` and is organized into four packages plus a command-line entry point and a Streamlit application:

Package	Responsibility
`owf/`	The water side. Parses EPANET `.inp` files, builds the WDN problem data and incidence matrices, forms the successive-linearization coefficients, assembles the CVXPY constraints/objective, runs the MILP loop, warm-starts and searches pump schedules, and validates the result against EPANET.
`pdn/`	The power side. Feeder datasets, the linear LinDistFlow/Kekatos voltage model, the nonlinear Z-bus power-flow validator, and a standalone reactive-power OPF.
`coupled/`	The joint C-OWPF. Augments the water model with the feeder voltage model, PV reactive support, and a linearized loss term; a voltage-aware pump-schedule search; and a two-simulator (EPANET + Z-bus) validator.
`ui/`	The Streamlit application (Home / Water / Power / Coupled / Guide tabs).
`main_owf.py`	Command-line driver for the decoupled OWF (`run_case`).
`app.py`	Streamlit entry point.

2 The solver stack

Both problems are nonconvex (Hazen–Williams head loss $\sim q^{1.852}$, FSP pump power, and the AC power flow). They are solved by **successive linear approximation**: the nonlinear terms are linearized about a current operating point, producing a mixed-integer linear/quadratic program that is solved by **HiGHS** (default) or **SCIP** (fallback) through **CVXPY**; the model is then relinearized about the new solution and re-solved until the iterate stops moving. Only the pump on/off variables are integer; the feeder (LinDistFlow + PV reactive) adds no new integers.

3 Core data structures

The problem data and results flow through a small number of dataclasses. Their full field lists appear in Part III; the summary below orients the reader.

Type	Module	Role
`RawNetwork`	`owf/epanet_io`	Static data read from an EPANET `.inp`: nodes, links, from/to, resistances, pump indices, tanks, reservoirs, junctions (0-based).
`Matrices`	`owf/connection_matrices`	Incidence / selection matrices (Π, Π', Λ, Θ, T, K, Ω, Δ, ...).
`WDN`	`owf/network`	The fully assembled water problem: matrices, pump/tank params, bounds, price, availability, EPANET ground truth. Passed to every water solve.
`LinPoint`	`owf/linearization`	Frozen linearization coefficients for one MILP solve (pipe/pump/power tangents).
`OWFResult`	`owf/solver`	Output of a water solve: flows, heads, on/off, pump power, cost, per-iteration error/objective, convergence, solver.
`LinDistModel`	`pdn/lindistflow`	Affine feeder voltage model $v^2 = R(-p) + X(-q) + V_k$ (all pu).
`PDN`	`pdn/network`	A feeder ready for coupling: its `LinDistModel`, PV/solar data, and voltage limits.
`CoupledConfig`	`coupled/config`	How a WDN and feeder are co-optimized (feeder key, pump→bus map, PV sizing, $v_{\min}/v_{\max}$, PF, loss on/off).
`CoupledResult`	`coupled/coupled_lp`	Output of a coupled solve: everything in `OWFResult` plus bus voltages, PV active/reactive, loss cost, total cost, voltage violation.

Part II. How the Problems Are Solved

4 Decoupled Optimal Water Flow (OWF, problem P3)

4.1 Entry point and inputs

A water case is run through `main_owf.run_case`:

```
run_case(net, mode, price, horizon, plot, outdir, verbose, solver='HIGHS')
    -> (CaseResult, wdn, result)
```

Inputs: `net` (network id, e.g. 8 = 8-node, 97 = Net3, 126 = BWSN); `mode` ∈ {`direct`, `warmstart`, `optimize`, `epanet`}; `price` (1 = time-of-use, 0 = flat); `horizon` (hours, or `None` for the network default); `solver`. **Output:** a `CaseResult` (costs, savings, EPANET-replay errors, timing), plus the `WDN` object and the `OWFResult`.

4.2 Building the problem: `owf.network.setup`

`run_case` first constructs a `SolverConfig` and calls `setup(config) -> WDN`, which: (i) reads the `.inp` via `epanet_io.read_inp -> RawNetwork`; (ii) runs an EPANET extended-period simulation (`run_epanet`) to obtain ground-truth flows/heads; (iii) builds the incidence matrices (`connection_matrices.build_matrices -> Matrices`); (iv) derives pump, tank, bound, price, and availability data (`_pump_params`, `_tank_params`, `_bounds`, `_price`, `_resolve_availability`). The returned `WDN` carries all of this and is the single object passed to every solver.

4.3 The successive-linearization loop: `owf.solver.solve_owf`

```
solve_owf(wdn, lin_override=None, eps_override=None) -> OWFResult
```

Steps:

1. **Initial point.** `initial_values.initial_point` returns a `LinPoint` (linearization tangents), the stacked iterate `eps` = [Heads; Flows; OnOff], and an initial `OnOff`. It can be seeded from EPANET flows/heads.
2. **Model.** `_build_model(wdn) -> Model` declares the CVXPY variables — nodal `Heads`, link `Flows`, binary `OnOff`, `Ppump`, and (in soft mode) bound slacks — and the linearization *parameters*.
3. **Constraints and objective.** `constraints.build_constraints(model, wdn)` assembles mass balance, reservoir/tank/junction head relations, the linearized pipe head loss (Π-based), the big-M pump curve, pump flow/power, and availability; `constraints.objective` sets $\min \sum_t \pi_t \sum_p P^{\text{pump}}_{p,t}/1000$.
4. **Iterate.** Each iteration `_set_params(model, lin)` injects the current `LinPoint`; the MILP is solved by HiGHS/SCIP; `linearization.linearize(flows, ...)` recomputes the tangents about the new flows; `error_norm` measures the change in `eps`. The loop stops at `tolerance` or `max_iter`.

Output — `OWFResult`: `flows` ($L \times T$), `heads` ($N \times T$), `onoff` ($P \times T$), `ppump` (linear) and `ppump_true` (nonlinear), scalar `cost`, per-iteration `errors`/`objective`, `converged`, `n_iter`, `solver`.

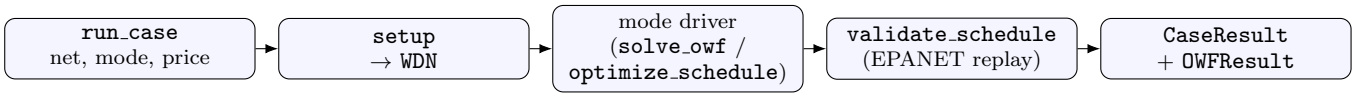
4.4 The four modes (how the schedule is obtained)

Mode	Driver (`owf/`)	What it does
`epanet`	`warmstart.solve_from_epanet`	Reproduces EPANET's rule-based operation (cost reference).
`direct`	`solver.solve_owf`	One successive-linearization run from the default initial point (radial networks).
`warmstart`	`warmstart.solve_warmstart` / `solve_multistart`	Converges from several EPANET-seeded schedules; robust on looped networks.
`optimize`	`warmstart.optimize_schedule`	Searches pump on/off (EPANET baseline → price-quantile candidates → fix-and-converge → 1-opt polish), returns savings vs EPANET.

Supporting routines: `epanet_default_onoff` (pump on/off implied by EPANET), `price_threshold_schedules` / `duty_preserving_schedule` (candidate schedules), `solve_fixed_schedule` / `_converge_fixed` (pin a schedule and converge the continuous problem), `true_energy_cost` (cost from the *nonlinear* FSP power).

4.5 Validation: `owf.validation.validate_schedule`

The optimized `onoff` is imposed back in EPANET (`epanet_io.simulate_with_schedule`) and the true hydraulics are compared to the model, yielding the maximum head/flow deviation and the minimum nodal pressure (feasibility). These populate `CaseResult.max_dhead` and `min_pressure`.



5 Coupled Optimal Water–Power Flow (C-OWPF, problem P2)

5.1 Entry point and inputs

The coupled problem is set up and solved through the `coupled/` package:

```
setup(net_num, cc, time=None, price_choice=1, solver='HIGHS') -> (WDN, PDN)
optimize_coupled_schedule(wdn, pdn, cc, ...) -> (CoupledResult, info)
```

Inputs: `net_num` (water network), a `CoupledConfig cc` (feeder key, pump→bus mapping, PV sizing $S_{pv} = k\,P_{pv}$, $v_{\min}/v_{\max}$, pump power factor, loss term on/off). `setup` builds the `WDN` (as for OWF) and the `PDN` via `pdn.network.PDN.build(feeder_key, ...)`, which wraps the affine `lindistflow.build_lindist -> LinDistModel` together with the feeder's PV/solar and voltage limits.

5.2 One coupled solve: `coupled.coupled_lp.solve_coupled`

```
build_coupled_problem(wdn, pdn, cc, trust_region=None) -> (problem, model, ctx)
solve_coupled(wdn, pdn, cc, lin_override=None, eps_override=None,
              trust_region=None) -> CoupledResult
```

`build_coupled_problem` takes the water `Model` and augments it with the power side:

- **Coupling.** The pump electrical power enters the feeder as a bus active load at the mapped bus; its reactive part is $q^{\text{pump}} = P^{\text{pump}}_{\text{pu}} \tan(\arccos \text{PF})$.
- **Voltages.** $v^2 = R(-p_{\text{net}}) + X(-q_{\text{net}}) + V_k$ (LinDistFlow), with voltage-dependent shunt capacitors folded into R, X, V_k.
- **PV reactive.** `_pv_capability` fixes PV active output and gives the exact reactive capability (octagon); `PV VAr` is dispatched to hold voltage.

- **Loss.** The branch-flow loss $\sum_\ell r_\ell(P_\ell^2+Q_\ell^2)$ is linearized each iteration via the subtree matrix `branch_flow_matrix`; the priced loss is added to the objective (paper 33d).
- **Voltage limits.** Enforced hard, or via the water side's soft-slack feasibility device.
- **Trust region (optional).** `trust_region=(z0, K)` caps the pump on/off Hamming distance from an incumbent `z0` to K flips.

The successive-linearization loop then proceeds exactly as in the water solve (relinearize → re-solve), and `_snapshot` reads the converged variables. **Output** — `CoupledResult`: `flows`, `heads`, `onoff`; `voltage` and `v2` ($N \times T$, pu); `pv_p`, `pv_q`; `energy_cost`, `loss_cost`, `total_cost`, `loss_kw`; `v_violation`; `water_max_slack`.

5.3 The voltage-aware schedule search: `coupled.schedule.optimize_coupled_schedule`

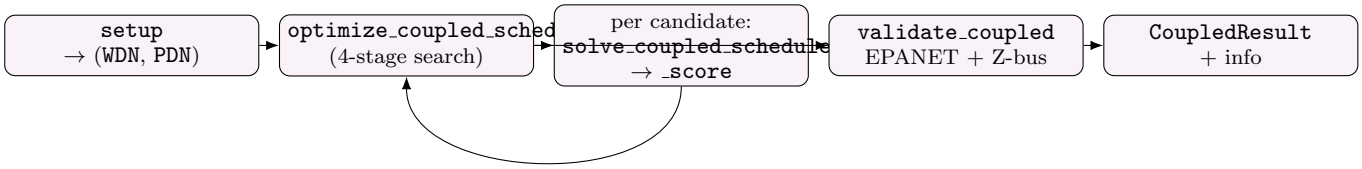
Unlike the water-only search, every candidate schedule is scored on the *coupled* solve, so a schedule that is cheap for water but causes an undervoltage is rejected. Stages:

1. **Baseline.** EPANET's own schedule, evaluated by `runner.solve_coupled_schedule` (fix `onoff`, warm-start from EPANET, converge the coupled LP).
2. **Candidates.** Price-quantile / load-shift schedules (`price_threshold_schedules`).
3. **Warm-started trust-region MILP.** Free binaries, warm-started at the incumbent's linearization, Hamming cap $K = \max(2, \lfloor n_p T/4 \rfloor)$ — the MATLAB-style free-binary MILP made reliable.
4. **1-opt polish.** Single pump-hour flips, greediest-price first.

Every candidate is ranked by `_score`: *coupled-feasible first* (water head slack and voltage violation within tolerance), then lowest true total cost (nonlinear pump energy + priced loss). A final clean re-solve of the winner *and* the EPANET baseline guarantees the coupled result is never worse than the decoupled one.

5.4 Validation: `coupled.validation.validate_coupled`

The winning solution is cross-checked against *both* exact simulators: the water schedule is replayed in EPANET (head/flow error, pressures), and the bus injections are replayed in the nonlinear Z-bus solver (`powerflow.zbus_solve`) for the true voltages and true $\sum |I|^2 r$ loss (`coupled_loss_kwh`).



6 Variable-Speed Pumps and Pressure-Reducing Valves

6.1 VSP (variable-speed pumps)

Marked per pump via `SolverConfig.vsp_pumps = {pump_id: (omega_min, omega_max)}` (water) or `CoupledConfig.vsp_pumps` (coupled). When any pump is a VSP, `_build_model` adds a continuous relative-speed variable `Speed` and a McCormick auxiliary `WW` ($\approx \omega f$; 4 envelope rows, the ω-lower-bound corner gated by on/off). The head gain becomes $h_0\omega^2 - \sigma f^\nu$, linearized as $-(C1M\,\omega + C2M\,f)$ with $C1M = -h_0\langle\omega\rangle$; the power constraint gains a `DPrime`·`WW` term (`DPrime`$= c_m h_0\langle\omega\rangle$, zero on FSP rows). Coefficients are relinearized at the current $(\langle f\rangle, \langle\omega\rangle)$ each iteration (`linearize(flows, M, pump, speed=...)`). Because the McCormick relaxation contracts the flow iterate slowly, convergence is also declared on a stable objective (`obj_rtol`); the fixed-speed path is bit-identical when no VSP is present. EPANET validation imposes the speed with `setLinkSettings` *after* `setLinkStatus` (opening a pump resets its setting), and the rule-based baseline reads back the speeds EPANET's own `SETTING` controls apply (`epanet_pump_speeds`). Result: on the 8-node, VSP at $\omega \in [0.8, 1]$ cuts cost ∼49% (decoupled) and ∼46% total (coupled, IEEE-13) while *lifting* the feeder voltage floor.

6.2 PRV (pressure-reducing valves)

Parsed from the `.inp` (`[VALVES]` type PRV; the psi setting via `getLinkInitialSetting`); overridable per valve with `SolverConfig.prv_settings = {valve_id: P_set_psi}`. The setpoint is $h^{\text{set}} = E_{\text{down}} + P^{\text{set}} \cdot 2.30724939$. Valves are excluded from the pipe energy equation (`connection_matrices`: `Pi_prime_valve`, up/down node selectors) but stay in the mass balance. `constraints.pressure_reducing_valves` implements the three-state model with two binaries: $x^{act} + x^{open} \leq 1$; big-M links $h_i - h_j = R^{PRV}$ when not closed; the downstream head is pinned to h^{set} when active; flow is gated by $x^{act} + x^{open}$; and $10^{-6}x^{act} \leq R^{PRV} \leq Mx^{act}$. The PRV is exact in the binaries – nothing is relinearized. Net 108 (`eightnode_prv`) reproduces EPANET's PRV regulation to $\sim$0.15 ft; in the coupled problem the optimally scheduled PRV avoids wasting pumped head, reducing pump power and hence feeder load, loss, and voltage stress. PRV decisions are exposed as `OWFResult.prv` / `CoupledResult.prv` and plotted by `ui/prv_plots.prv_panel`.

6.3 Related coupled drivers

`runner.solve_coupled_schedule` (fixed-schedule coupled solve, used as the scoring oracle); `runner.solve_coupled_epanet` (coupled solve on EPANET's own schedule); `coupled_lp.default_pump_buses` (auto-place pumps at the electrically weakest buses); `opf.solve_pdn_opf` (the standalone power-only reactive OPF used by the Power tab).

Part III. Complete Per-Module API Reference

Auto-generated from the source by an AST walk. For each function: its signature (inputs) and return annotation (outputs) followed by its docstring. For each class: its fields and methods.

7 Entry points

7.1 main_owf.py

Module purpose. Entry point for the FSP Optimal Water Flow (OWF) solver.

_fallbacks

```
def _fallbacks(solver: str) -> tuple:
    """Fall back to HiGHS then SCIP (whichever are installed), skipping the primary."""
```

_warmstart_config

```
def _warmstart_config(net: int, price: int, horizon, solver: str) -> SolverConfig:
```

class CaseResult

```
class CaseResult:
    # fields:
    label: str
    net_num: int
    mode: str
    price: str
    horizon: int
    elapsed: float
    converged: bool
    n_iter: int
    epanet_cost: float = float('nan')
    owf_cost: float = float('nan')
    savings_pct: float = float('nan')
    max_dhead: float = float('nan')
    max_dpumpflow: float = float('nan')
    min_pressure: float = float('nan')
    solver: str = 'HIGHS'
    note: str = ''
    plots: list = field(default_factory=list)
```

run_case

```
def run_case(net: int, mode: str, price: int, horizon, plot: bool, outdir: str, verbose: bool, solver: str='HIGHS
     ', vsp: dict=None, prv: dict=None, teach: bool=False) -> tuple[Optional[CaseResult], object, object]:
    """Construct and solve one case; returns (CaseResult, wdn, result).

    ``vsp`` maps a pump id to its (omega_min, omega_max) speed bounds; listed
    pumps run at variable speed. When any VSP is present the case uses a
    soft-bound, damped direct solve (the McCormick relaxation needs it).
    ``prv`` maps a valve id to its pressure setting P_set (psi), overriding the
    .inp value (h_set = downstream elevation + P_set * 2.3072)."""
```

comparison_table

```
def comparison_table(cases: list) -> str:
    """EPANET(rules) vs C-OWF comparison for one or more cases."""
```

_ask

```
def _ask(prompt: str, default: str, valid=None) -> str:
```

interactive

```
def interactive() -> None:
```

main

```
def main() -> None:
```

7.2 app.py

Module purpose. C-OWPF Streamlit app – coupled Optimal Water-Power Flow.

7.3 guide.py

Module purpose. Methodology / user guide rendered inside the Streamlit app (Guide tab).

render_guide

```
def render_guide(st) -> None:
```

8 owf – Water Distribution Network & OWF solver

8.1 owf/__init__.py

Module purpose. FSP Optimal Water Flow (OWF) – Python translation of the MATLAB C-OWPF WDN solver.

__getattr__

```
def __getattr__(name):
```

8.2 owf/config.py

Module purpose. Configuration for the FSP Optimal Water Flow (OWF) solver.

class NetworkSpec

```
class NetworkSpec:
    """Everything network-specific needed to build the OWF problem.

``pump_coefficients`` is an optional override: one ``[h0, r, v]`` row per pump
giving the pump head-gain curve  H_gain(q) = h0 - r * q**v.  When ``None``
(the default) the coefficients are derived from the EPANET pump head curve,
which keeps the optimizer consistent with EPANET and avoids the value drift
seen in the hard-coded ``Prepare_net_WDN.m`` tables."""
    # fields:
    net_num: int
    name: str
    inp_relpath: str
    pump_coefficients: Optional[List[List[float]]] = None
    bypasses: Optional[dict] = None
    pump_availability: Optional[dict] = None
```

method inp_path

```
def inp_path(self) -> Path:
```

class SolverConfig

```
class SolverConfig:
    """Successive-linearization loop and MILP solver settings."""
    # fields:
    net_num: int = 8
    time: Optional[int] = None
    price_choice: int = 1
    choice: int = 1
    tol: float = 0.5
    max_iter: int = 50
    big_m: Optional[float] = None
    mass_balance_warmup: bool = False
    mass_balance_warmup_iters: int = 10
    damping: float = 1.0
    fixed_schedule: Optional['np.ndarray'] = None
    vsp_pumps: Optional[dict] = None
    prv_settings: Optional[dict] = None
    fixed_speed: Optional['np.ndarray'] = None
    record_iterates: bool = False
    soft_bounds: bool = False
    penalty_weight: float = 1000.0
    penalty_growth: float = 2.0
    penalty_max: float = 10000000.0
    feas_tol: float = 0.001
    obj_rtol: float = 0.001
    solver: str = 'HIGHS'
    fallback_solvers: tuple = ('SCIP',)
    verbose: bool = False
    data_dir: Path = field(default=DATA_DIR)
```

method `spec`

```
def spec(self) -> NetworkSpec:
```

available_solvers

```
def available_solvers() -> list[str]:
    """MILP solvers from SOLVER_CHOICES that are actually installed."""
```

8.3 owf/epanet_io.py

Module purpose. EPANET input parsing and hydraulic ground-truth via epyt.

class RawNetwork

```
class RawNetwork:
    """Static network data extracted from an EPANET .inp file (0-based indices)."""
    # fields:
    link_name_id: list[str]
    link_diameter: np.ndarray
    link_length: np.ndarray
    link_roughness: np.ndarray
    link_resistance: np.ndarray
    link_minor_resistance: np.ndarray
    link_type: list[str]
    link_pump_index: np.ndarray
    link_pump_count: int
    link_valve_index: np.ndarray
    link_setting: np.ndarray
    pump_coefficients: np.ndarray
    closed_pipe_index: np.ndarray
    node_index: np.ndarray
    junction_index: np.ndarray
    reservoir_index: np.ndarray
    tank_index: np.ndarray
    node_elevations: np.ndarray
    node_name_id: list
    node_x: np.ndarray
```

```
    node_y: np.ndarray
    from_node: np.ndarray
    to_node: np.ndarray
    tank_diameter: np.ndarray
    tank_min_level: np.ndarray
    tank_max_level: np.ndarray
    tank_init_level: np.ndarray
    tank_area: np.ndarray
    inp_path: str
    handle: object
```

method `n_nodes`

```
def n_nodes(self) -> int:
```

method `n_links`

```
def n_links(self) -> int:
```

_derive_pump_coefficients

```
def _derive_pump_coefficients(d, n_pumps: int) -> np.ndarray:
    """Derive pump curve coefficients [h0, r, v=2] from EPANET head curves.

For a single design point (Qd, Hd) EPANET's convention gives
``h0 = 1.3333 Hd`` and ``r = h0 / (2 Qd)**2`` (so H_gain = h0 - r q^2 passes
through (0, 1.3333 Hd), (Qd, Hd) and (2 Qd, 0)); multi-point curves are fit
to H = h0 - r Q^v.

Pumps with no head curve (EPANET curve index 0 -- e.g. constant-power pumps)
cannot be modeled by this FSP head-gain form; they yield NaN coefficients and
must be supplied via ``NetworkSpec.pump_coefficients``."""
```

read_inp

```
def read_inp(inp_path) -> RawNetwork:
    """Load an EPANET .inp file and extract static OWF data (ports read_inp.m)."""
```

simulate_with_schedule

```
def simulate_with_schedule(inp_path, pump_links_1based, onoff: np.ndarray, time: int, n_nodes: int, n_links: int,
      bypass_links=None, pump_speeds: np.ndarray=None) -> tuple[np.ndarray, np.ndarray]:
    """Impose a pump on/off schedule in EPANET and return the true hydraulics.

Deletes existing controls/rules, drives each pump's status hour-by-hour from
``onoff`` (Pu x T), runs the extended-period simulation, and returns
(heads, flows) shaped (N x time), (L x time) sampled at hour boundaries.
Shared by the schedule-imposed validation and the EPANET warm-start.

``bypass_links`` is an optional list of ``(link_1based, pump_pos)`` pairs for
switched bypasses: each is opened exactly when its pump is off. Without this,
deleting the controls would leave a bypass stuck at its initial status and the
simulation would be meaningless (e.g. Net3's pipe 330).

``pump_speeds`` (Pu x T, optional) imposes each pump's relative speed for
variable-speed pumps; when given, a running pump's EPANET setting is set to its
scheduled speed (fixed-speed pumps use speed 1)."""
```

read_controls_rules

```
def read_controls_rules(inp_path) -> dict:
    """Return the operational logic EPANET uses, straight from the .inp file:
the [CONTROLS] (simple) and [RULES] (rule-based) sections as text lines.

This is the *baseline* schedule the C-OWPF cost is compared against."""
```

epanet_pump_speeds

```
def epanet_pump_speeds(raw: RawNetwork, time: int) -> np.ndarray:
    """Pump relative speeds EPANET applies under the network's own controls.

Reads the link *Setting* time series from EPANET's native extended-period run
(``getComputedTimeSeries().Setting``) for the pump links, so that a network
whose ``.inp`` schedules variable speed -- via ``[CONTROLS] LINK p SETTING s``,
``[RULES]``, or a speed pattern -- has an honest baseline. This is convention-
agnostic: EPANET reports the *applied* relative speed however the control was
written. Returns (n_pumps x time); off hours (setting ~ 0) are reported as 1.0
(the pump flow is zero there, so the value does not affect energy)."""
```

run_epanet

```
def run_epanet(raw: RawNetwork) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """Extended-period hydraulic simulation (ports init_epanet.m).

Returns (flows, heads, headloss, demand) each shaped (n_steps, n_links|n_nodes),
i.e. time along axis 0 -- matching the raw EPANET output ordering the MATLAB
code transposes downstream.

Loads a fresh EPANET handle from the source .inp so it stays robust even after
other simulations (e.g. the warm-start) have opened and unloaded handles."""
```

8.4 owf/connection_matrices.py

Module purpose. Topology / incidence matrices for the WDN (ports ConnectionMatrices_WDN.m).

class Matrices

```
class Matrices:
    # fields:
    Pi: np.ndarray
    Pi_telda: np.ndarray
    Pi_prime: np.ndarray
    Pi_reduced: np.ndarray
    Lambda: np.ndarray
    Theta: np.ndarray
    Tau: np.ndarray
    Kappa: np.ndarray
    Omega: np.ndarray
    Delta: np.ndarray
    pipe_index: np.ndarray
    bypass_index: np.ndarray
    Pi_telda_bypass: np.ndarray
    Pi_prime_bypass: np.ndarray
    Omega_bypass: np.ndarray
    S_bypass_pump: np.ndarray
    valve_index: np.ndarray
    Pi_prime_valve: np.ndarray
    valve_up_sel: np.ndarray
    valve_down_sel: np.ndarray
```

build_matrices

```
def build_matrices(raw: RawNetwork, bypass_index: np.ndarray=None, bypass_pump_pos: np.ndarray=None, valve_index:
      np.ndarray=None) -> Matrices:
```

8.5 owf/network.py

Module purpose. WDN assembly (ports WDN_setup_IEEE_ACCESS.m + definebounds_WDN.m).

class TankParams

```
class TankParams:
    # fields:
    area: np.ndarray
    init_head: np.ndarray
    min_head: np.ndarray
    max_head: np.ndarray
    mean_head: np.ndarray
    del_tk_tanks: np.ndarray
    Atk: np.ndarray
    Btk: np.ndarray
```

class Bounds

```
class Bounds:
    # fields:
    min_nodal_heads: np.ndarray
    max_nodal_heads: np.ndarray
```

class WDN

```
class WDN:
    # fields:
    config: SolverConfig
    spec: cfg.NetworkSpec
    raw: RawNetwork
    M: Matrices
    pump: PumpParams
    tank: TankParams
    bounds: Bounds
    time: int
    price_final: np.ndarray
    junction_demand_profile: np.ndarray
    lin0: LinPoint
    int_eps: np.ndarray
    int_onoff: np.ndarray
    pump_avail: dict = None
    valve_hset: np.ndarray = None
    valve_fmax: np.ndarray = None
```

method n_nodes

```
def n_nodes(self) -> int:
```

method n_links

```
def n_links(self) -> int:
```

method n_pumps

```
def n_pumps(self) -> int:
```

method n_pipes

```
def n_pipes(self) -> int:
```

method n_tanks

```
def n_tanks(self) -> int:
```

method n_reservoirs

```
def n_reservoirs(self) -> int:
```

method n_junctions

```
def n_junctions(self) -> int:
```

method `n_vsp`

```
def n_vsp(self) -> int:
```

method `n_valves`

```
def n_valves(self) -> int:
```

_pump_params

```
def _pump_params(raw: RawNetwork, spec: cfg.NetworkSpec) -> PumpParams:
```

_tank_params

```
def _tank_params(raw: RawNetwork, time: int) -> TankParams:
```

_bounds

```
def _bounds(raw: RawNetwork, tank: TankParams, time: int) -> Bounds:
```

_price

```
def _price(config: SolverConfig, time: int) -> np.ndarray:
```

_resolve_bypasses

```
def _resolve_bypasses(raw: RawNetwork, spec: cfg.NetworkSpec):
    """Map spec.bypasses {bypass_id: pump_id} to link indices + pump positions."""
```

_resolve_vsp

```
def _resolve_vsp(raw: RawNetwork, pump: PumpParams, config: SolverConfig) -> None:
    """Mark which pumps are variable-speed (in place) from ``config.vsp_pumps``.

``config.vsp_pumps`` maps a pump link id to its (omega_min, omega_max) bounds.
Unlisted pumps stay fixed-speed (omega == 1), i.e. the FSP model."""
```

_resolve_valves

```
def _resolve_valves(raw: RawNetwork, config: SolverConfig=None) -> tuple:
    """PRV link indices and downstream head setpoints.

A PRV regulates its downstream junction to ``h_set = E_down + P_set``, where
``E_down`` is the downstream node elevation and ``P_set`` is the valve setting
converted from psi to ft (paper: h^set = E_j + P^set).
``config.prv_settings`` ({valve_id: psi}) overrides the .inp settings."""
```

_resolve_availability

```
def _resolve_availability(raw: RawNetwork, spec: cfg.NetworkSpec) -> dict:
    """Map spec.pump_availability {pump_id: (start,end)} to {pump_pos: (start,end)}."""
```

setup

```
def setup(config: SolverConfig) -> WDN:
    """Build the full WDN problem data for the given configuration."""
```

8.6 `owf/initial_values.py`

Module purpose. Initial linearization point (ports Initial_Values_WDN.m).

initial_point

```
def initial_point(raw: RawNetwork, M: Matrices, pump: PumpParams, bounds, time: int, choice: int, flows_ep: np.
    ndarray, heads_ep: np.ndarray) -> tuple[LinPoint, np.ndarray, np.ndarray]:
    """Return (initial LinPoint, stacked iterate int_eps, initial OnOff).

``flows_ep``/``heads_ep`` are EPANET's computed time series (steps x L|N),
supplied by the caller so EPANET is only run once."""
```

8.7 owf/linearization.py

Module purpose. Successive-linearization coefficients (ports CalculateNewIterationValues.m).

class PumpParams

```
class PumpParams:
    # fields:
    h0: np.ndarray
    r_m: np.ndarray
    v_m: np.ndarray
    c_m: float
    max_flow: np.ndarray
    is_vsp: np.ndarray = None
    omega_min: np.ndarray = None
    omega_max: np.ndarray = None
```

method __post_init__

```
def __post_init__(self):
```

method any_vsp

```
def any_vsp(self) -> bool:
```

class LinPoint

```
class LinPoint:
    """Frozen linearization coefficients for one MILP solve."""
    # fields:
    Cp: np.ndarray
    C1M: np.ndarray
    C2M: np.ndarray
    APrimePump: np.ndarray
    BPrimePump: np.ndarray
    Cp_bypass: np.ndarray = None
    DPrime: np.ndarray = None
```

linearize

```
def linearize(flows: np.ndarray, M: Matrices, pump: PumpParams, speed: np.ndarray=None) -> LinPoint:
    """Build linearization coefficients around a flow field ``flows`` (L x T).

``speed`` (Pu x T relative pump speed) is used only for variable-speed pumps;
when ``None`` it defaults to 1 (fixed speed), which recovers the FSP model."""
```

stack_eps

```
def stack_eps(heads: np.ndarray, flows: np.ndarray, onoff: np.ndarray) -> np.ndarray:
    """Concatenate [Heads; Flows; OnOff] (the successive-linearization iterate)."""
```

error_norm

```
def error_norm(eps: np.ndarray, prev_eps: np.ndarray) -> float:
    """Euclidean norm of the change in the stacked iterate."""
```

8.8 owf/constraints.py

Module purpose. OWF constraints in CVXPY (ports the define*_CVX.m modules, one per function).

reservoir_head

```
def reservoir_head(model, wdn):
    """Theta H == Theta H_min  (fix reservoir heads to their elevation)."""
```

tank_flow_aux

```
def tank_flow_aux(model, wdn):
    """TankFlow_aux == Tau' (-Pi) Flows  (net inflow to each tank)."""
```

tank_state_space

```
def tank_state_space(model, wdn):
    """Hdummy_i = A_tk H0_i + (del/A_i) Btk TankFlow_aux_i  (integrator)."""
```

tank_hdummy_head

```
def tank_hdummy_head(model, wdn):
    """Tau' H == [H0 , Hdummy(:,1:T-1)]  (tank head = previous dummy level)."""
```

tank_head_bounds

```
def tank_head_bounds(model, wdn):
    """H_min <= tank heads <= H_max (expressed over Tau)."""
```

pipe_head_loss

```
def pipe_head_loss(model, wdn):
    """Pi_telda H == Cp + Pi_prime Q  (linearized Hazen-Williams)."""
```

mass_balance

```
def mass_balance(model, wdn):
    """Pi_reduced Q == -demand  (conservation of mass at junctions)."""
```

closed_pipes_zero

```
def closed_pipes_zero(model, wdn):
    """Permanently-closed pipes carry no flow."""
```

switched_bypasses

```
def switched_bypasses(model, wdn):
    """Bypass pipes that are OPEN iff their pump is OFF.

Let z = OnOff of the controlling pump (via S). When z=1 (pump on) the bypass
is closed: zero flow, head loss relaxed. When z=0 (pump off) the bypass is a
normal pipe: flow free, Hazen-Williams head loss enforced."""
```

pump_availability

```
def pump_availability(model, wdn):
    """Force pumps off outside their source-availability window."""
```

pump_flow

```
def pump_flow(model, wdn):
    """q_min OnOff <= Lambda Q <= q_max OnOff  (flow only when pump is on)."""
```

_head_gain

```
def _head_gain(model, wdn, pump_flows):
    """Linearized pump head gain C1M(+ *omega) + C2M f.

FSP: C1M + C2M f (C1M = -h0).  VSP: C1M*omega + C2M f (C1M = -h0<omega>).
The two forms coincide for FSP because there omega is pinned to OnOff and
C1M = -h0, so C1M*OnOff = -h0 when the pump runs (and the bound is relaxed off)."""
```

pump_negative_headloss

```
def pump_negative_headloss(model, wdn):
    """Head gain is nonnegative:  C1M(*omega) + C2M (Lambda Q) <= 0."""
```

pump_power

```
def pump_power(model, wdn):
    """Ppump == A'(Lambda Q) + B' OnOff (+ D' WW for VSP)  (linearized pump power)."""
```

pump_bigm

```
def pump_bigm(model, wdn):
    """Big-M pump head-gain curve, enforced only when OnOff == 1."""
```

vsp_speed_mccormick

```
def vsp_speed_mccormick(model, wdn):
    """Variable-speed pump: speed bounds gated by on/off, and the McCormick
envelope of the bilinear WW = omega * (pump flow).

Applies to every pump when a VSP is present. FSP pumps have omega_min =
omega_max = 1, so the speed bounds pin omega = OnOff and the envelope pins
WW = pump flow exactly -- leaving their power (DPrime = 0) unchanged."""
```

pressure_reducing_valves

```
def pressure_reducing_valves(model, wdn):
    """Three-state PRV model via two binaries (paper eq. PRVoperation).

x_act (active/regulating) and x_open (fully open) with x_act + x_open <= 1
(both zero -> closed). h_set is the downstream head setpoint; R_prv is the
valve head loss. Big-M switches the three states:
  * active (y=1): downstream head pinned to h_set, upstream >= h_set, R_prv > 0;
  * open   (z=1): upstream <= h_set, zero head loss (h_up = h_down);
  * closed (0,0): zero flow, up/down heads decoupled."""
```

junction_head_bounds

```
def junction_head_bounds(model, wdn):
    """H_min <= junction heads <= H_max."""
```

tank_terminal

```
def tank_terminal(model, wdn):
    """Hdummy(:,end) >= tank mean head  (terminal level target)."""
```

junction_head_bounds_soft

```
def junction_head_bounds_soft(model, wdn):
```

tank_head_bounds_soft

```
def tank_head_bounds_soft(model, wdn):
```

tank_terminal_soft

```
def tank_terminal_soft(model, wdn):
```

build_constraints

```
def build_constraints(model, wdn, include_mass_balance: bool=True, soft: bool=False):
    """Assemble the full FSP OWF constraint set (optionally with soft head bounds)."""
```

_energy_cost

```
def _energy_cost(model, wdn):
```

objective

```
def objective(model, wdn):
    """min  sum_t price(t) * sum_pumps(Ppump)/1000   (energy cost)."""
```

objective_soft

```
def objective_soft(model, wdn):
    """Energy cost plus a penalty on head-bound slacks (penalty CCP warm-start)."""
```

8.9 owf/solver.py

Module purpose. Successive linear-approximation loop (ports WDN_OWF_IEEEACCESS_cvx.m).

class Model

```
class Model:
    """CVXPY decision variables and linearization parameters."""
    # fields:
    Flows: cp.Variable
    Heads: cp.Variable
    Ppump: cp.Variable
    Hdummy: cp.Variable
    OnOff: cp.Variable
    TankFlow_aux: cp.Variable
    Cp: cp.Parameter
    C1M: cp.Parameter
    C2M: cp.Parameter
    APrime: cp.Parameter
    BPrime: cp.Parameter
    s_jlo: Optional[cp.Variable] = None
    s_jhi: Optional[cp.Variable] = None
    s_tlo: Optional[cp.Variable] = None
    s_thi: Optional[cp.Variable] = None
    s_term: Optional[cp.Variable] = None
    penalty: Optional[cp.Parameter] = None
    Cp_bypass: Optional[cp.Parameter] = None
    Speed: Optional[cp.Variable] = None
    WW: Optional[cp.Variable] = None
    DPrime: Optional[cp.Parameter] = None
    x_act: Optional[cp.Variable] = None
    x_open: Optional[cp.Variable] = None
    R_prv: Optional[cp.Variable] = None
```

class OWFResult

```
class OWFResult:
    # fields:
    status: str
    n_iter: int
    converged: bool
    objective: float
    heads: np.ndarray
    flows: np.ndarray
    onoff: np.ndarray
    ppump_linear: np.ndarray
    ppump_true: np.ndarray
    speed: np.ndarray = None
    prv: dict = None
    errors: list = field(default_factory=list)
    objectives: list = field(default_factory=list)
    max_slack: float = float('nan')
    iterates: list = field(default_factory=list)
```

_build_model

```
def _build_model(wdn: WDN) -> Model:
```

_set_params

```
def _set_params(model: Model, lin: LinPoint) -> None:
```

_max_slack

```
def _max_slack(model: Model) -> float:
```

_prv_snapshot

```
def _prv_snapshot(model: Model) -> Optional[dict]:
    """Current PRV decisions (x_act / x_open / R_prv) as plain arrays, or None."""
```

_true_pump_power

```
def _true_pump_power(wdn: WDN, flows: np.ndarray, speed: np.ndarray=None) -> np.ndarray:
    """Nonlinear pump power at a flow solution.

FSP: c_m (h0 - r f^v) f.  VSP (with relative speed omega): c_m (h0 omega^2 - r f^v) f."""
```

solve_owf

```
def solve_owf(wdn: WDN, lin_override: Optional[LinPoint]=None, eps_override: Optional[np.ndarray]=None) ->
     OWFResult:
    """Run the successive-linearization loop.

``lin_override`` / ``eps_override`` let a caller seed the initial linearization
and iterate from an arbitrary warm-start point (e.g. EPANET flows under a
chosen pump schedule) instead of ``wdn.lin0`` / ``wdn.int_eps``."""
```

8.10 owf/warmstart.py

Module purpose. EPANET-based multi-start warm-starting for the successive approximation.

epanet_default_onoff

```
def epanet_default_onoff(wdn: WDN, thresh: float=0.001) -> np.ndarray:
    """Pump on/off implied by EPANET's own (rule-based) operation."""
```

candidate_schedules

```
def candidate_schedules(wdn: WDN) -> dict[str, np.ndarray]:
    """A small library of pump on/off schedules to warm-start from."""
```

warmstart_point

```
def warmstart_point(wdn: WDN, onoff: np.ndarray):
    """Linearization point + stacked iterate from EPANET flows under ``onoff``."""
```

_score

```
def _score(r: OWFResult) -> tuple:
    """Rank results: converged first, then smaller bound violation, then cheaper."""
```

solve_multistart

```
def solve_multistart(wdn: WDN, schedules: Optional[dict[str, np.ndarray]]=None, verbose: bool=True) -> tuple[
     OWFResult, str, dict[str, OWFResult]]:
    """Run the successive approximation from each candidate EPANET warm-start.

Returns (best_result, best_name, all_results). Best = converged, then least
head-bound violation, then lowest energy cost."""
```

true_energy_cost

```
def true_energy_cost(wdn: WDN, flows: np.ndarray, speed: np.ndarray=None) -> float:
    """Energy cost using the TRUE nonlinear pump power (not the linearized model).

This is the honest basis for comparing two schedules -- the per-iteration
objective uses linearized power, which differs slightly between linearization
points. ``speed`` (Pu x T) applies the variable-speed power form on VSP pumps."""
```

_converge_fixed

```
def _converge_fixed(wdn: WDN, onoff: np.ndarray, seed_flows: np.ndarray, seed_heads: np.ndarray, max_iter: int
     =20) -> OWFResult:
    """Fix a schedule and converge the continuous problem from a given seed."""
```

_apply_availability

```
def _apply_availability(wdn: WDN, sched: np.ndarray) -> np.ndarray:
    """Zero out hours where a pump's source is unavailable."""
```

duty_preserving_schedule

```
def duty_preserving_schedule(wdn: WDN) -> np.ndarray:
    """Load-shift EPANET's operation: keep each pump's total on-hours but move
them to the cheapest hours.

The price-quantile candidates run pumps *fewer* hours (only the cheapest),
which under-pumps and drains the tanks below their minimum. Preserving each
pump's EPANET duty and merely re-timing it to cheap hours keeps the tanks in
bounds while still cutting the time-of-use bill -- the feasible way to save on
high-duty-cycle systems (e.g. BWSN's 11 h / 21 h pumps)."""
```

price_threshold_schedules

```
def price_threshold_schedules(wdn: WDN) -> dict[str, np.ndarray]:
    """Candidate schedules that run pumps only during the cheapest hours.

A family of price quantiles (plus all-on and a duty-preserving load-shift) --
physically motivated for time-of-use pricing and cheap to enumerate."""
```

optimize_schedule

```
def optimize_schedule(wdn: WDN, inner_iter: Optional[int]=None, feas_tol: float=2.0, polish: bool=True, max_flips
     : int=30, verbose: bool=True) -> tuple[OWFResult, dict]:
    """Optimize the pump on/off schedule and report savings vs EPANET.

Each candidate schedule is evaluated **honestly**: fix it, converge the
continuous hydraulics, and score the TRUE (nonlinear) energy cost plus the
head-bound violation. We do not let the frozen linearization *predict* a
distant schedule's quality -- that is what makes a single free-binary MILP
return the incumbent (its coefficients simply don't apply far away).

Stages:
  1. **Baseline** -- EPANET's own operation.
  2. **Candidates** -- price-quantile schedules (run only during the cheapest
     hours), all-on, and a MILP proposal at the incumbent's linearization.
  3. **Polish** (optional) -- 1-opt: flip single pump-hours of the winner and
     keep any flip that lowers cost, giving a schedule that is locally optimal
     w.r.t. single-hour changes.

Ranking: feasible (slack <= feas_tol) first, then lowest true cost.

Returns (best result, info dict with baseline/best cost, savings and trace)."""
```

solve_from_epanet

```
def solve_from_epanet(wdn: WDN) -> OWFResult:
    """Reproduce EPANET's own operation: seed the linearization from EPANET's
computed hydraulics, fix that pump schedule, and converge the continuous
problem. Robust for multi-pump / switched-bypass networks (e.g. Net3) where
EPANET's controls already give a valid, feasible operating point."""
```

solve_fixed_schedule

```
def solve_fixed_schedule(wdn: WDN, onoff: np.ndarray) -> OWFResult:
    """Pin the pump on/off schedule and converge the continuous problem.

With the binaries fixed the per-iteration problem is a pure LP, so the
successive linearization converges reliably (as on the tree networks)."""
```

solve_warmstart

```
def solve_warmstart(wdn: WDN, schedules: Optional[dict[str, np.ndarray]]=None, verbose: bool=True) -> tuple[
     OWFResult, str]:
    """Two-phase warm-start solve for hard networks.

Phase 1: multi-start over candidate EPANET schedules to pick a good binary
pump schedule. Phase 2: fix that schedule and converge the continuous problem.
Returns (result, best_schedule_name)."""
```

8.11 owf/validation.py

Module purpose. Validate the OWF solution against EPANET hydraulics (ports the tail of Main_OWF_IEEE_ACCESS.m): error norms on heads, pipe flows and pump flows, plus a true-power comparison.

class ValidationReport

```
class ValidationReport:
    # fields:
    error_heads: float
    error_pipe_flows: float
    error_pump_flows: float
    pump_power_epanet: np.ndarray
    pump_power_opt_true: np.ndarray
    heads_epanet: np.ndarray
    flows_epanet: np.ndarray
```

method summary

```
def summary(self) -> str:
```

validate

```
def validate(wdn: WDN, result: OWFResult) -> ValidationReport:
```

class ScheduleValidationReport

```
class ScheduleValidationReport:
    """Result of imposing the optimized pump schedule back into EPANET
(reproduces the paper's Fig. 4 hydraulic-feasibility check)."""
    # fields:
    max_abs_head: float
    max_abs_pipe_flow: float
    max_abs_pump_flow: float
    norm_head: float
    norm_pipe_flow: float
    norm_pump_flow: float
    heads_epanet: np.ndarray
    flows_epanet: np.ndarray
```

method `summary`

```
def summary(self) -> str:
```

validate_schedule

```
def validate_schedule(wdn: WDN, result: OWFResult) -> ScheduleValidationReport:
    """Fix the optimized pump on/off schedule into EPANET, re-simulate the true
nonlinear hydraulics, and compare against the optimizer's heads/flows.

Unlike :func:`validate` (which compares against EPANET's own rule-based
operation), this drives EPANET with the *optimized* schedule, so a good
linearization should reproduce EPANET closely."""
```

8.12 `owf/plots.py`

Module purpose. Diagnostic plots: convergence, EPANET-vs-OWF flows/heads, and pump schedules.

_plt

```
def _plt():
```

plot_convergence

```
def plot_convergence(result: OWFResult):
    """Successive-linearization error and objective per iteration."""
```

plot_pump_schedule

```
def plot_pump_schedule(wdn: WDN, result: OWFResult):
    """Optimized pump on/off schedule against the electricity price."""
```

plot_flows

```
def plot_flows(wdn: WDN, result: OWFResult, report: ScheduleValidationReport, max_pipes: int=6):
    """Pump flows and a sample of pipe flows: OWF vs EPANET (imposed schedule)."""
```

plot_heads

```
def plot_heads(wdn: WDN, result: OWFResult, report: ScheduleValidationReport, max_junctions: int=5):
    """Tank and junction heads: OWF vs EPANET (imposed schedule)."""
```

plot_error_summary

```
def plot_error_summary(wdn: WDN, result: OWFResult, report: ScheduleValidationReport):
    """Per-time-step max |error| in heads and flows vs EPANET."""
```

plot_all

```
def plot_all(wdn: WDN, result: OWFResult, report: Optional[ScheduleValidationReport]=None, outdir: str | Path='
    outputs', prefix: Optional[str]=None) -> list[Path]:
    """Write the full plot set to ``outdir``; returns the written paths."""
```

8.13 owf/netmap.py

Module purpose. Interactive network-map view (Plotly) for the Streamlit UI.

extract_map_data

```
def extract_map_data(wdn, result=None) -> dict:
    """Collect everything the map needs from a WDN (+ optional OWFResult)."""
```

build_map_figure

```
def build_map_figure(data: dict, hour: int | None=None):
    """Plotly figure of the network; colored by solution state at ``hour``."""
```

build_animated_map_figure

```
def build_animated_map_figure(data: dict):
    """Animated map: a Play button steps through the hours, drawing each link's
flow as a direction arrow (angle = flow direction, size = |flow|) over the
junction-pressure field. Falls back to the static map if no solution."""
```

9 pdn – Power Distribution Network models

9.1 pdn/__init__.py

Module purpose. Power distribution network (PDN) side of C-OWPF.

9.2 pdn/feeders.py

Module purpose. Distribution-feeder data for the power side of C-OWPF.

9.3 pdn/feeders_paper.py

Module purpose. Two SCE feeders transcribed from the paper's Table I (Fig. 5, 47-bus) and Table II (Fig. 6, 56/57-bus).

_build

```
def _build(label, desc, edges, loads, pv_mw, caps_mvar, vbase, sbase, v0_kv, virtual_slack_load=None) -> dict:
```

9.4 pdn/network.py

Module purpose. PDN object: a feeder's linear voltage model plus its DER / limit settings.

class PDN

```
class PDN:
    """A distribution feeder ready for the coupled optimization (all pu)."""
    # fields:
    key: str
    label: str
    model: LinDistModel
    pv_rating: np.ndarray
    solar_profile: np.ndarray
    vmin: float = DEFAULT_VMIN
    vmax: float = DEFAULT_VMAX
    orig_id: list = field(default_factory=list)
```

method N

```
def N(self) -> int:
```

method pv_buses

```
def pv_buses(self) -> np.ndarray:
```

method limit_pv

```
def limit_pv(self, k: int) -> 'PDN':
    """Keep only the ``k`` largest-rated PV sites active (zero the rest)."""
```

method solar

```
def solar(self, T: int) -> np.ndarray:
    """Solar availability tiled/truncated to a T-hour horizon."""
```

method build

```
def build(key: str, pv_sizing: float=1.2, vmin: float=DEFAULT_VMIN, vmax: float=DEFAULT_VMAX) -> 'PDN':
    """Construct a PDN from the feeder registry.

Each PV inverter's apparent-power rating is ``pv_sizing`` x the bus's
nominal active load (the paper places PV on load-hosting buses).  Sizing
strictly to load -- not a fixed per-unit floor -- keeps PV proportional to
the feeder regardless of its VA base, avoiding artificial reverse-flow
overvoltage on high-base feeders (e.g. IEEE-33 at 100 MVA)."""
```

9.5 pdn/lindistflow.py

Module purpose. Linear (LinDistFlow / Kekatos) distribution-network voltage model.

class LinDistModel

```
class LinDistModel:
    """Prebuilt affine voltage model for one feeder (all pu)."""
    # fields:
    N: int
    A_tilde: np.ndarray
    F: np.ndarray
    R: np.ndarray
    X: np.ndarray
    V_k: np.ndarray
    r: np.ndarray
    x: np.ndarray
    p_load: np.ndarray
    q_load: np.ndarray
    b_shunt: np.ndarray
    pv_mask: np.ndarray
    v0_sq: float
    parent: np.ndarray
```

method v2

```
def v2(self, p_net: np.ndarray, q_net: np.ndarray) -> np.ndarray:
    """Squared bus voltages for given net loads (pu).

``q_net`` is the net reactive load EXCLUDING shunt caps -- the caps are
voltage-dependent (q*v) and already folded into ``R``/``X``/``V_k`` via the
Kekatos (I  X*diag(q)) correction, matching paper eq. (1b)/(2b)."""
```

method `voltage`

```
def voltage(self, p_net: np.ndarray, q_net: np.ndarray) -> np.ndarray:
```

build_lindist

```
def build_lindist(feeder: dict) -> LinDistModel:
    """Assemble the affine voltage model from a ``pdn.feeders`` entry."""
```

branch_flow_matrix

```
def branch_flow_matrix(model: LinDistModel) -> np.ndarray:
    """Subtree-membership matrix T (N x N): ``P_branch = T @ p_net``.

Row i (the branch feeding bus i) sums the net injections of bus i and all its
descendants -- the lossless LinDistFlow branch flow. With per-unit branch flows
the network loss is ``sum_i r_i (P_branch_i^2 + Q_branch_i^2)``."""
```

9.6 pdn/powerflow.py

Module purpose. Nonlinear single-phase Z-bus (fixed-point) power flow – the *validator*.

zbus_solve

```
def zbus_solve(model: LinDistModel, p_net: np.ndarray, q_net: np.ndarray, tol: float=1e-10, max_iter: int=1000):
    """Full nonlinear Z-bus solve.

Returns ``(vmag, vcomplex, loss)``: non-slack voltage magnitudes (pu), their
complex values, and the total real network loss (pu) from the exact branch
currents ``I_br = y (A_tilde V_full)`` as ``sum |I_br|^2 r``.

``p_net`` / ``q_net`` are net loads in pu (positive = consumption) with PV and
pump loads already folded in, but **excluding** shunt caps -- those are added
here as a Y-bus susceptance (the physically-correct, voltage-dependent model)."""
```

zbus_powerflow

```
def zbus_powerflow(model: LinDistModel, p_net: np.ndarray, q_net: np.ndarray, tol: float=1e-10, max_iter: int
    =1000) -> np.ndarray:
    """Nonlinear bus voltage magnitudes (pu) -- thin wrapper over ``zbus_solve``."""
```

9.7 pdn/opf.py

Module purpose. Standalone distribution-network reactive-power OPF.

class PDNOPFResult

```
class PDNOPFResult:
    # fields:
    feeder: str
    q_pv: np.ndarray
    p_pv: np.ndarray
    pv_buses: np.ndarray
    v_lin: np.ndarray
    v_nl: np.ndarray
    v_nl_base: np.ndarray
    loss_nl: np.ndarray
    loss_base: np.ndarray
    p_net: np.ndarray
    q_net: np.ndarray
    v_violation: float
```

```
    SBase: float
```

method `loss_kw`

```
def loss_kw(self) -> np.ndarray:
```

method `loss_base_kw`

```
def loss_base_kw(self) -> np.ndarray:
```

method `loss_reduction_pct`

```
def loss_reduction_pct(self) -> float:
```

pump_load_to_bus

```
def pump_load_to_bus(pdn: PDN, ppump_kw: np.ndarray, pump_bus, scale: float=1.0):
    """Map pump electrical power (Pu x T, kW) to a per-bus active load (N x T, pu)."""
```

solve_pdn_opf

```
def solve_pdn_opf(pdn: PDN, T: int, pump_load_pu: np.ndarray | None=None, load_shape: np.ndarray | None=None,
     vmin: float=0.95, vmax: float=1.05, soft_voltage: bool=True, voltage_penalty: float=10000.0, solver: str='
     HIGHS', verbose: bool=False) -> PDNOPFResult:
    """Dispatch PV reactive to hold voltage, then verify with the nonlinear Z-bus."""
```

10 coupled – Coupled C-OWPF solver

10.1 coupled/__init__.py

Module purpose. Coupled Optimal Water-Power Flow (C-OWPF).

10.2 coupled/config.py

Module purpose. Configuration for the coupled Optimal Water-Power Flow (C-OWPF).

class CoupledConfig

```
class CoupledConfig:
    """How a water network and a distribution feeder are co-optimized."""
    # fields:
    feeder: str = 'ieee13'
    pump_bus: Optional[Sequence[int]] = None
    vsp_pumps: Optional[dict] = None
    prv_settings: Optional[dict] = None
    pump_load_scale: float = 1.0
    load_scale: float = 1.0
    load_profile: Optional[np.ndarray] = None
    enable_pv: bool = True
    pv_sizing: float = 1.2
    pump_pf: float = 0.9
    include_loss: bool = True
    vmin: float = 0.95
    vmax: float = 1.05
    soft_voltage: bool = True
    voltage_penalty: float = 10000.0
```

method `load_shape`

```
def load_shape(self, T: int) -> np.ndarray:
```

10.3 coupled/coupled_lp.py

Module purpose. Coupled Optimal Water-Power Flow (C-OWPF) solver.

class CoupledResult

```
class CoupledResult:
    # fields:
    status: str
    n_iter: int
    converged: bool
    objective: float
    energy_cost: float
    heads: np.ndarray
    flows: np.ndarray
    onoff: np.ndarray
    ppump_true: np.ndarray
    water_max_slack: float
    voltage: np.ndarray
    v2: np.ndarray
    pv_p: np.ndarray
    pv_q: np.ndarray
    pv_buses: np.ndarray
    p_net: np.ndarray
    q_net: np.ndarray
    grid_kw: np.ndarray
    pump_bus: np.ndarray
    v_min: float
    v_violation: float
    loss_kw: np.ndarray = None
    loss_cost: float = float('nan')
    total_cost: float = float('nan')
    speed: np.ndarray = None
    prv: dict = None
    ppump_linear: np.ndarray = None
    errors: list = field(default_factory=list)
    objectives: list = field(default_factory=list)
```

default_pump_buses

```
def default_pump_buses(pdn: PDN, n_pumps: int) -> np.ndarray:
    """Place pumps at the feeder's electrically weakest (lowest nominal-voltage)
buses, where a pump load most stresses the network -- the interesting case."""
```

_pv_capability

```
def _pv_capability(pdn: PDN, cc: CoupledConfig, T: int):
    """PV active output (fixed at availability) and the exact reactive capability.

PV *active* p_pv[k,t] = rating[k] * solar(t) is fixed (the paper controls PV
reactive, not active).  With active known, the smart-inverter capability circle
|q| <= sqrt(S^2 - p^2) is a CONSTANT bound per (bus, hour) -- exact and LP-safe."""
```

build_coupled_problem

```
def build_coupled_problem(wdn: WDN, pdn: PDN, cc: CoupledConfig, trust_region: Optional[tuple]=None):
    """Construct the augmented CVXPY problem; returns (problem, model, ctx).

``trust_region=(z0, K)`` -- only meaningful when the schedule is free (MILP) --
limits the Hamming distance of the pump on/off variable to at most ``K`` flips
from an incumbent schedule ``z0``, keeping the warm-started linearization valid."""
```

_feeder_sbase

```
def _feeder_sbase(pdn: PDN) -> float:
```

solve_coupled

```
def solve_coupled(wdn: WDN, pdn: PDN, cc: CoupledConfig, lin_override: Optional[LinPoint]=None, eps_override:
     Optional[np.ndarray]=None, trust_region: Optional[tuple]=None) -> CoupledResult:
    """Run the coupled successive-linearization loop and return a CoupledResult."""
```

_failed_result

```
def _failed_result(status, n_iter, ctx, errors, objectives) -> CoupledResult:
    """Return a clearly-failed CoupledResult (e.g. infeasible hard voltage)."""
```

_snapshot

```
def _snapshot(wmodel, wdn, pdn, cc, ctx) -> dict:
    """Read current variable values into plain numpy (safe to store)."""
```

10.4 coupled/runner.py

Module purpose. High-level drivers for the coupled water-power optimization.

setup

```
def setup(net_num: int, cc: CoupledConfig, time: Optional[int]=None, price_choice: int=1, solver: str='HIGHS') ->
     tuple[WDN, PDN]:
    """Build the water network and the distribution feeder for a coupled run."""
```

solve_coupled_schedule

```
def solve_coupled_schedule(wdn: WDN, pdn: PDN, cc: CoupledConfig, onoff: np.ndarray, soft_bounds: bool=True,
    max_iter: int=20, replay_polish: bool=False) -> CoupledResult:
    """Fix ``onoff``, warm-start from EPANET, and converge the coupled problem.

``replay_polish=True`` (VSP networks only) finishes with the EPANET
speed-pinned polish -- the same fix the decoupled path uses -- so the
returned heads/flows replay in EPANET at the solved speeds."""
```

_replay_polish

```
def _replay_polish(wdn: WDN, pdn: PDN, cc: CoupledConfig, res: CoupledResult, rounds: int=2) -> CoupledResult:
    """EPANET speed-pinned polish for a coupled VSP result.

The damped homotopy can stop at a point where the McCormick relaxation gap
leaves the model's pump flow far from what EPANET's affinity-scaled curve
delivers at the solved speeds (tens of ft / hundreds of GPM of replay
error). Mirror of the decoupled fix in ``main_owf``: replay (schedule,
speeds) in EPANET, re-converge the COUPLED problem with the schedule AND
speeds pinned (with omega fixed, WW = omega*f is exact) linearized at the
replay point, and keep whichever candidate -- including the original --
replays best. Monotone by construction."""
```

solve_coupled_epanet

```
def solve_coupled_epanet(wdn: WDN, pdn: PDN, cc: CoupledConfig, replay_polish: bool=False) -> CoupledResult:
    """Coupled solve reproducing EPANET's own (rule-based) pump schedule."""
```

10.5 coupled/schedule.py

Module purpose. Coupled pump-schedule optimization (voltage-aware).

_score

```
def _score(wdn, res: CoupledResult, feas_tol: float, v_tol: float):
    """Rank key: (infeasible?, total violation, total coupled cost pump+loss)."""
```

optimize_coupled_schedule

```
def optimize_coupled_schedule(wdn, pdn, cc, v_tol: float=0.02, feas_tol: float=2.0, inner_iter: int=15, trust_k:
    int | None=None, polish: bool=True, max_flips: int=20, verbose: bool=True, use_milp: bool=True, candidates:
    dict | None=None):
    """Search for the pump schedule minimizing coupled cost + voltage feasibility.

``candidates`` overrides the default price-quantile candidate set (pass a small
dict to cut runtime on big networks); ``use_milp=False`` skips the trust-region
MILP (the most expensive stage on a large feeder + many-pump network).
```

```
Returns (best CoupledResult, info dict)."""
```

10.6 coupled/validation.py

Module purpose. Validate a coupled solution against BOTH ground-truth simulators.

coupled_loss_kwh

```
def coupled_loss_kwh(pdn: PDN, res: CoupledResult) -> float:
    """Total true (nonlinear Z-bus) real network loss over the horizon, in kW*h."""
```

class CoupledValidationReport

```
class CoupledValidationReport:
    # fields:
    water: ScheduleValidationReport
    v_lin: np.ndarray
    v_nl: np.ndarray
    v_err_max: float
    v_err_mean: float
    vmin_nl: float
    v_violation_nl: float
```

method summary

```
def summary(self) -> str:
```

validate_coupled

```
def validate_coupled(wdn: WDN, pdn: PDN, res: CoupledResult, vmin: float=0.95, vmax: float=1.05) ->
     CoupledValidationReport:
    """Cross-check a coupled result against EPANET (water) and Z-bus (power)."""
```

11 ui – Streamlit application

11.1 ui/__init__.py

Module purpose. UI building blocks for the C-OWPF Streamlit app.

11.2 ui/theme.py

Module purpose. Shared look-and-feel for the C-OWPF app: appearance modes, CSS, headers, cards.

_surface_css

```
def _surface_css(p: dict) -> str:
    """CSS rules that force one palette on the app surfaces (no media query)."""
```

_register_plotly

```
def _register_plotly(mode: str) -> None:
    """Give every Plotly chart a theme-neutral font so axes read on any background."""
```

inject_theme

```
def inject_theme(st, mode: str='System') -> None:
    """Inject base CSS + the chosen appearance mode; tune Plotly fonts to match."""
```

inject_css

```
def inject_css(st) -> None:
```

hero

```
def hero(st, title: str, subtitle: str, tagline: str='') -> None:
```

section_header

```
def section_header(st, color: str, title: str, subtitle: str='') -> None:
```

11.3 ui/landing.py

Module purpose. Landing page: paper intro, the C-OWPF solve flowchart, a successive-linearization animation with a "why it's better" explainer, the networks, and key formulations.

_method_note_download

```
def _method_note_download(st) -> None:
    """Offer the one-page 'how each solve mode is derived in code' transparency note."""
```

_flowchart_svg

```
def _flowchart_svg() -> str:
    """Detailed flowchart of how the coupled C-OWPF problem is solved."""
```

successive_approx_fig

```
def successive_approx_fig():
    """Animate successive linearization converging on the nonlinear head-loss curve."""
```

_networks_table

```
def _networks_table():
```

render_landing

```
def render_landing(st) -> None:
```

11.4 ui/water_section.py

Module purpose. Water tab: the decoupled FSP Optimal Water Flow (direct / warmstart / optimize / EPANET modes), schedules, EPANET validation, plus per-session comparison and diff.

_csv_bytes

```
def _csv_bytes(df: pd.DataFrame) -> bytes:
```

_pump_ids

```
def _pump_ids(net: int) -> list:
    """Pump link ids for a water net (cached; used to pick variable-speed pumps)."""
```

_prv_ids

```
def _prv_ids(net: int) -> list:
    """PRV link ids for a water net (cached)."""
```

_build_exports

```
def _build_exports(case, wdn, result) -> dict:
```

_session_df

```
def _session_df(records) -> pd.DataFrame:
```

_render_inspector

```
def _render_inspector(st, rec) -> None:
    """Pick any links/nodes and plot their time series -- flow(t), head(t),
pressure(t) -- straight from the solved fields (student exploration)."""
```

_render_pump_curves

```
def _render_pump_curves(st, rec) -> None:
    """EPANET-style pump curve H(q) = h0*omega^2  r*q^v with the run's operating
points overlaid (colored by hour)."""
```

_render_constraints

```
def _render_constraints(st, rec) -> None:
    """Teaching mode: per-iteration constraint residuals/margins with the math."""
```

_render_case

```
def _render_case(st, rec) -> None:
```

_controls

```
def _controls(st):
```

render_water

```
def render_water(st) -> None:
```

11.5 ui/power_section.py

Module purpose. Power tab: standalone distribution-network reactive OPF with DER controls.

_feeder_choice_labels

```
def _feeder_choice_labels():
```

_epanet_pump_kw

```
def _epanet_pump_kw(net: int):
    """EPANET (rule-based) pump electrical power for a water net.

Returns ``(ppump, ids, Tw)`` at the water net's OWN horizon ``Tw`` -- the PDN
OPF then runs over exactly that horizon so pump load and feeder load line up."""
```

_owf_pump_kw

```
def _owf_pump_kw(net: int, solver: str):
    """C-OWF *optimized* pump electrical power for a water net.

Solves the decoupled water problem in the network's RECOMMENDED mode (the
same one the Water tab suggests) and returns the optimized schedule's true
nonlinear pump power -- the optimized decoupled hand-off."""
```

render_power

```
def render_power(st) -> None:
```

_render_power_result

```
def _render_power_result(st) -> None:
```

11.6 ui/coupled_section.py

Module purpose. Coupled tab: joint water-power C-OWPF vs the decoupled hand-off, cost compared.

_probe_pump_ids

```
def _probe_pump_ids(net: int):
    """Pump ids for a water net (cached so switching widgets doesn't re-run EPANET)."""
```

render_coupled

```
def render_coupled(st) -> None:
```

_render_solution_tables

```
def _render_solution_tables(st, res, R, prefix: str, title: str) -> None:
    """Full decision variables of one solution: schedule (with speeds), pump power,
flows and heads -- as labeled tables with CSV downloads."""
```

_render_coupled_inspector

```
def _render_coupled_inspector(st, cpl, val, R, fmeta) -> None:
    """Pick feeder buses / lines / water elements and plot their time series:
V(t) linear vs Z-bus, line P(t) & Q(t), water flow(t) and head(t)."""
```

_render_coupled_result

```
def _render_coupled_result(st) -> None:
```

11.7 ui/pdn_plots.py

Module purpose. Plotly figures for the power-distribution side (voltages, PV VAr, loss, map).

feeder_layout

```
def feeder_layout(feeder: dict):
    """Layered left-to-right layout of a radial feeder from its parent array.

Returns (x, y) over GLOBAL nodes 0..N (0 = slack) and the edge list."""
```

feeder_map

```
def feeder_map(feeder: dict, v_nl: np.ndarray, hour: int, pv_buses, pump_buses, vmin=0.95, vmax=1.05):
    """One-line feeder diagram, buses colored by nonlinear voltage at ``hour``."""
```

voltage_profile

```
def voltage_profile(v_lin: np.ndarray, v_nl: np.ndarray, hour: int, vmin=0.95, vmax=1.05, orig_id=None):
    """Per-bus voltage at one hour: linear (LinDistFlow) vs nonlinear (Z-bus)."""
```

voltage_heatmap

```
def voltage_heatmap(v_nl: np.ndarray, vmin=0.95, vmax=1.05, title='Nonlinear voltage |V| (pu)'):
    """Bus x hour heatmap of the true (Z-bus) voltage magnitude."""
```

pv_reactive_chart

```
def pv_reactive_chart(q_pv: np.ndarray, p_pv: np.ndarray, pv_buses, orig_id=None):
    """PV reactive setpoints (stacked) and total active, over the day."""
```

loss_chart

```
def loss_chart(loss_base_kw: np.ndarray, loss_opt_kw: np.ndarray):
    """True network loss per hour: no-reactive baseline vs optimized setpoints."""
```

coupled_network_map

```
def coupled_network_map(water_md: dict, feeder: dict, pump_bus, pump_ids, pv_buses=None, valve_links=None,
     show_node_labels: bool=True, show_bus_labels: bool=True, show_coupling_labels: bool=True):
    """Side-by-side depiction of the interconnected energy systems (paper Fig. 1
style): the WDN on the left, the feeder on the right, and dashed pump->bus
coupling lines (the Xi matrix) between them."""
```

pv_capacity_animation

```
def pv_capacity_animation(p_pv: np.ndarray, q_pv: np.ndarray, rating: np.ndarray, labels: list):
    """Animated per-hour inverter loading: ||(p,q)|| / S_max per PV site.

One bar per site;  Play steps through the hours. The 100% line is the
inverter apparent-power rating S -- the capability circle the OPF respects."""
```

11.8 ui/prv_plots.py

Module purpose. Paper-style PRV figures (Plotly): valve state timeline, downstream head vs the setpoint h_set, valve head loss R_prv, and valve flow – model vs EPANET where available. Mirrors the manuscript's PRV accuracy/status figures.

prv_states

```
def prv_states(prv: dict) -> np.ndarray:
    """(Nv x T) state code: 0 = closed, 1 = open, 2 = active."""
```

prv_panel

```
def prv_panel(wdn, heads: np.ndarray, flows: np.ndarray, prv: dict, heads_ep: np.ndarray=None, flows_ep: np.
    ndarray=None, heads_rule: np.ndarray=None, flows_rule: np.ndarray=None, valve_pos: int=0):
    """2x2 PRV panel for one valve (paper Fig. 4-style, plus the state timeline).

``heads_ep``/``flows_ep``: EPANET replay of the OPTIMIZED schedule (validation).
``heads_rule``/``flows_rule``: EPANET running its OWN rule-based controls -- a
DIFFERENT schedule, overlaid so the pressure regulation the rules achieve can
be compared with the optimized regulation (and cost differences explained:
the rules hold the .inp setting, the optimizer holds the user's h_set)."""
```

11.9 ui/constraint_view.py

Module purpose. Teaching mode: per-iteration constraint-satisfaction viewer.

iteration_report

```
def iteration_report(wdn, snap: dict) -> list[dict]:
    """One row per constraint family: worst residual (=) or worst violation (<=)."""
```

evolution

```
def evolution(wdn, iterates: list[dict]) -> dict:
    """{family: [worst per iteration]} for the evolution plot."""
```

evolution_figure

```
def evolution_figure(wdn, iterates: list[dict]):
    """Log-scale line plot: each family's worst residual/violation vs iteration."""
```

11.10 ui/correlations.py

Module purpose. Correlation explorer: how the system's signals co-evolve over the day.

_norm

```
def _norm(v: np.ndarray) -> np.ndarray:
```

_overlay_fig

```
def _overlay_fig(sigs: dict, sel: list):
    """Normalized overlay of the selected signals with a playable hour cursor."""
```

_corr_fig

```
def _corr_fig(sigs: dict, sel: list):
    """Pearson correlation heatmap of the selected signals."""
```

_pair_fig

```
def _pair_fig(sigs: dict, xname: str, yname: str):
    """Scatter of one signal against another, colored by hour, with trend + r."""
```

render_correlations

```
def render_correlations(st, sigs: dict, key: str) -> None:
    """The full explorer: overlay + matrix + user-drawn pair scatter."""
```

11.11 ui/capture.py

Module purpose. OS-level stdout/stderr capture for solver logs.

_retarget_log_handlers

```
def _retarget_log_handlers(new_stream, old_streams):
    """Point plain logging StreamHandlers at ``new_stream`` for the capture.

CVXPY's MOSEK interface emits the solver log via ``logging`` (not stdout).
Those handlers hold a reference to the ORIGINAL console stream, which under
a served app (run_ui.bat) can have an invalid Windows handle -- every MOSEK
line then dumps an "OSError: [WinError 6]" logging-error traceback. Retarget
them into the captured stream (so MOSEK logs land in the Solver-log tab) and
return the originals for restoration. FileHandlers are left untouched."""
```

capture_fds

```
def capture_fds():
    """Context manager capturing fd-level stdout+stderr.

Besides redirecting fd 1/2 into a temp file, ``sys.stdout``/``sys.stderr``
are REBOUND to fresh UTF-8 writers over the captured descriptors for the
duration of the block. This matters in a served Streamlit app: the server's
original ``sys.stdout`` can be a closed/odd console stream (run_ui.bat,
detached windows, cp1252 consoles), and a bare ``print`` inside the block
would crash the tab. With the rebind, every print -- ours or a library's --
lands in the captured log regardless of the host console's state.

Usage::

    with capture_fds() as cap:
        ...solves / prints...
    text = cap["text"]          # available after the block exits"""
```

run_capturing_all

```
def run_capturing_all(fn):
    """Run ``fn`` capturing fd-level output; returns (fn_result, captured_text)."""
```

11.12 ui/pump_power.py

Module purpose. Shared "optimized vs true pump power" check panel.

pump_power_check

```
def pump_power_check(st, ppump_linear, ppump_true, pump_ids=None, key: str='pp', label: str='Pump power check')
    -> None:
    """Expander comparing _t optimizer (linear) vs _t true pump power."""
```

12 tests

12.1 tests/test_coupled.py

Module purpose. Coupled water-power (C-OWPF) tests.

test_lindist_matches_nonlinear_nominal

```
def test_lindist_matches_nonlinear_nominal(key):
    """Linear LinDistFlow voltages match the nonlinear Z-bus at nominal load."""
```

test_coupled_fixed_schedule_converges_and_validates

```
def test_coupled_fixed_schedule_converges_and_validates(feeder):
```

test_pv_reactive_supports_voltage

```
def test_pv_reactive_supports_voltage():
    """PV reactive control must lift the minimum voltage vs the no-PV case."""
```

test_coupled_schedule_optimization_saves

```
def test_coupled_schedule_optimization_saves():
    """The voltage-aware schedule search must beat EPANET on cost, stay feasible,
and its winner must validate against the nonlinear Z-bus."""
```

test_objective_is_pump_cost_only

```
def test_objective_is_pump_cost_only():
    """The reported energy cost must equal the water-only pump cost (feeder-agnostic)."""
```

12.2 tests/test_eightnode.py

Module purpose. Regression tests for the 8-node FSP OWF problem.

solved

```
def solved():
```

test_network_sizes

```
def test_network_sizes():
```

test_converges

```
def test_converges(solved):
```

test_onoff_binary

```
def test_onoff_binary(solved):
```

test_mass_balance_holds

```
def test_mass_balance_holds(solved):
    """Pi_reduced Q == -demand at junctions (a model equality constraint)."""
```

test_reservoir_head_fixed

```
def test_reservoir_head_fixed(solved):
    """Reservoir head is pinned to its elevation."""
```

test_pump_flow_within_bounds

```
def test_pump_flow_within_bounds(solved):
```

test_pump_off_implies_zero_flow

```
def test_pump_off_implies_zero_flow(solved):
    """When OnOff == 0 the pump flow must be (near) zero."""
```

test_power_matches_epanet_aggregate

```
def test_power_matches_epanet_aggregate(solved):
    """Total true FSP power should be in the right ballpark vs EPANET."""
```

12.3 tests/test_more_networks.py

Module purpose. Tests for the additional FSP networks: 3-node (working) and Net1 (loads).

threenode

```
def threenode():
```

test_threenode_sizes

```
def test_threenode_sizes(threenode):
```

test_threenode_pump_coeff_from_epanet

```
def test_threenode_pump_coeff_from_epanet(threenode):
    """Pump coefficients are derived from the EPANET large-pump curve (~533, ~1.3e-6)."""
```

test_threenode_converges

```
def test_threenode_converges(threenode):
```

test_threenode_matches_epanet_schedule

```
def test_threenode_matches_epanet_schedule(threenode):
    """Optimized schedule re-run in EPANET matches the optimizer closely (Fig. 4)."""
```

test_net1_builds

```
def test_net1_builds():
```

test_net1_default_init_does_not_converge

```
def test_net1_default_init_does_not_converge():
    """Documents the limitation the warm-start solves."""
```

test_net1_warmstart_converges

```
def test_net1_warmstart_converges():
    """EPANET multi-start warm-start converges Net1 and matches EPANET (Fig. 4)."""
```

test_net2_builds

```
def test_net2_builds():
```

test_net2_warmstart_converges

```
def test_net2_warmstart_converges():
```

test_ky3_not_registered

```
def test_ky3_not_registered():
    """KY3 is archived (pumps carry no EPANET head curve) -- not a live network."""
```

test_net3_structure

```
def test_net3_structure():
```

test_net3_reproduces_epanet

```
def test_net3_reproduces_epanet():
    """solve_from_epanet reproduces EPANET's operation on Net3 (with the bypass)."""
```

12.4 tests/test_optimize_schedule.py

Module purpose. Tests for pump-schedule optimization (savings vs EPANET's own operation).

optimized_net2

```
def optimized_net2():
```

test_price_threshold_family_shape

```
def test_price_threshold_family_shape():
```

test_true_cost_positive

```
def test_true_cost_positive():
```

test_net2_optimization_saves

```
def test_net2_optimization_saves(optimized_net2):
    """The optimizer should beat EPANET's operation on Net2, feasibly."""
```

test_net2_optimized_schedule_feasible_in_epanet

```
def test_net2_optimized_schedule_feasible_in_epanet(optimized_net2):
    """Re-simulating the optimized schedule in EPANET gives positive pressures
and in-bounds tanks -- i.e. the savings are physically real."""
```

test_availability_respected

```
def test_availability_respected():
    """Net3's Lake pump must stay off outside its availability window."""
```

12.5 tests/test_plots.py

Module purpose. Smoke tests for the diagnostic plots.

solved

```
def solved():
```

test_individual_plots

```
def test_individual_plots(solved):
```

test_plot_all_writes_files

```
def test_plot_all_writes_files(solved, tmp_path):
```